

AI Agent の実運用でなぜガバナンスが必要か

AI Agent の特性とリスク

- ・ ツール・社内データへアクセス
- ・ 実システムの操作
- ・ 複数ACTIONを自律的に実行

プロンプトだけでは実行時の振る舞いを保証できない

生じ得る主なリスク

誤操作 / 権限逸脱 / 情報漏洩 / Prompt Injection

法規制等によるガバナンス要求

例1

California SB 53

Transparency in Frontier Artificial Intelligence Act

- ・ 安全性フレームワークの策定・運用
- ・ 重大な安全インシデントの報告
- ・ 透明性・説明責任の強化

例2

EU AI Act

- ・ リスク管理
- ・ 人による監督
- ・ ログ記録・追跡可能性の確保
- ・ サイバーセキュリティ

※求められる義務は、AIの分類や事業者の立場によって異なる

AI Agent のリスクと法規制等の要求を、実運用上のガバナンスへ落とし込むことが重要

Policy

ポリシーに基づく行動・権限制御

Audit

操作や判断の監査（記録・追跡）

AI Agent の実運用でなぜガバナンスが必要か

AI Agent の特性とリスク

- ・ ツール・社内データへアクセス
- ・ 実システムの操作
- ・ 複数ACTIONを自律的に実行

プロンプトだけでは実行時の振る舞いを保証できない

生じ得る主なリスク

誤操作 / 権限逸脱 / 情報漏洩 / Prompt Injection

法規制等によるガバナンス要求

例1

California SB 53

Transparency in Frontier Artificial Intelligence Act

- ・ 安全性フレームワークの策定・運用
- ・ 重大な安全インシデントの報告
- ・ 透明性・説明責任の強化

例2

EU AI Act

- ・ リスク管理
- ・ 人による監督
- ・ ログ記録・追跡可能性の確保
- ・ サイバーセキュリティ

※求められる義務は、AIの分類や事業者の立場によって異なる

AI Agent のリスクと法規制等の要求を、実運用上のガバナンスへ落とし込むことが重要

Policy

ポリシーに基づく行動・権限制御

Audit

操作や判断の監査（記録・追跡）

02 AI AgentのActionはどのように実行されるか

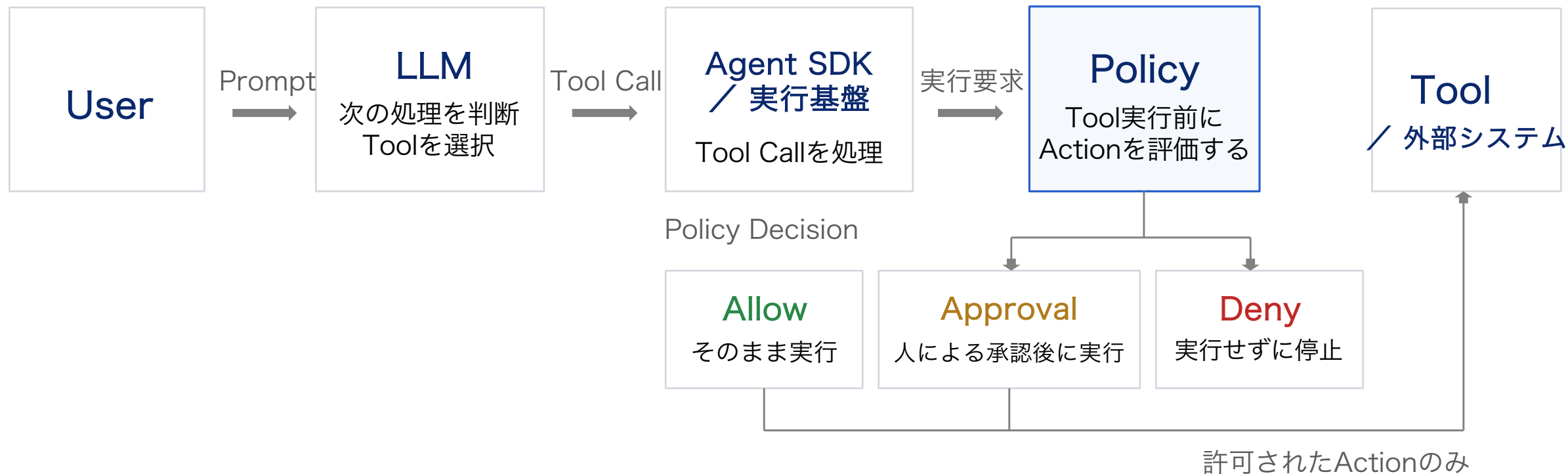


LLMの判断は、Tool Callを通じて実システムのActionにつながる

03 Promptによる指示と実行時のPolicy制御

Promptの例 (Userが入力する自然言語の指示)

「重要なファイルは変更・削除しないで」



PromptはAgentに期待する行動を指示し, Policyは生成されたActionを実行してよいか判断する

04 Agent SDK固有の制御と共通Policy適用

SDK固有の制御

OpenAI Agents SDK

Guardrails / Approval

Claude Agent SDK

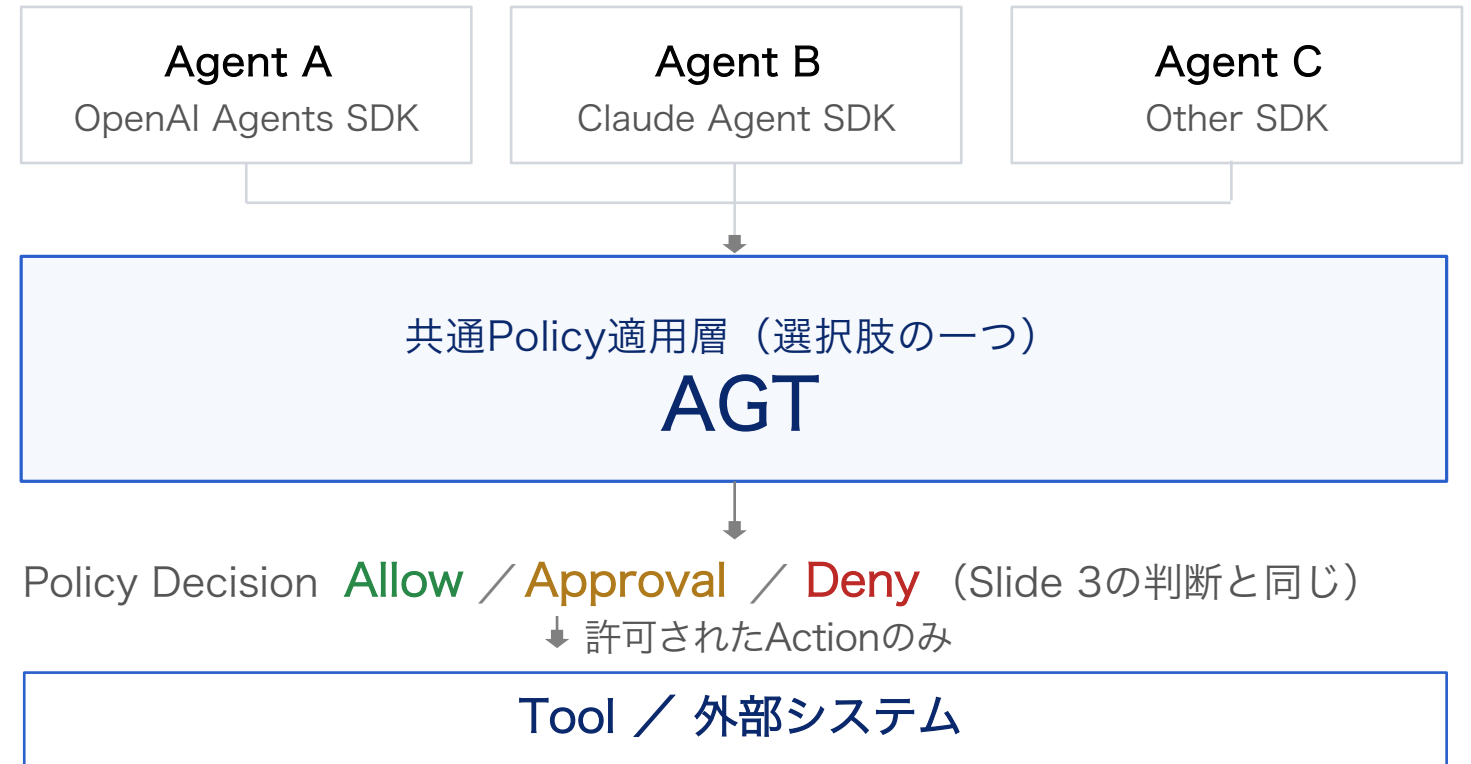
Permissions / Hooks

単一Agent / SDKでは、SDK固有機能でActionを制御できる場合がある

複数SDKへ展開すると、
同じ業務Policyが各実装へ分散する可能性

→ 共通Policy適用層という選択肢を検討する

複数Agent / SDKを横断する共通Policy適用



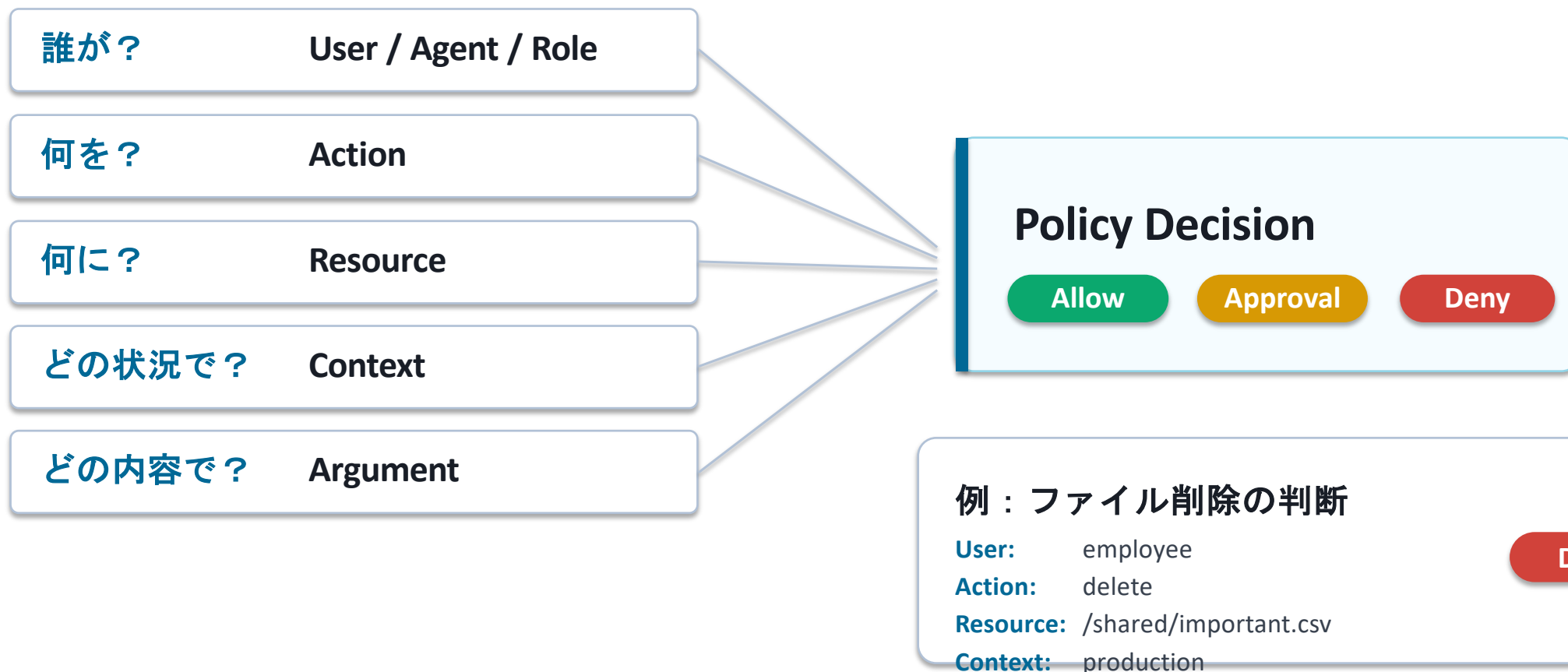
Policyの一元管理 / 判断基準の統一 / Agent・Tool追加時の再利用

※共通化できる範囲とSDK差異の吸収方法は検討対象

SDK固有の制御を置き換えるのではなく、複数Agent / SDK / Toolにまたがる共通Policy管理層としてAGTを検討する

5. Policyは何を見て判断するか

Tool名だけでなく、「誰が・何を・何に・どの状況で」実行するかを判断材料にする



別軸の制御：Token / Cost / Rate / Budget などの利用量もPolicy対象になり得る

共通Policyとして利用する情報を，SDK差異を吸収できる形式へ揃えることが論点になる

6. Policyをどのように表現・評価するか

PolicyをAgent実装から分離し，外部で管理・評価する構成を検討する



PolicyをAgent実装から分離し，共通のPolicy Engineで評価できる構成を検討する

7. 今回の検証：ファイル操作へのPolicy適用

単純なファイル操作で、Action取得 → 共通Policy評価 → 実行フロー制御の基本動作を確認する



3つのテストケース

read_file()

Allow

→ 実行

write_file()

Approval

→ 人が承認後に実行

delete_file()

Deny

→ 実行しない

確認項目：① Actionを実行前に取得 ② 共通形式でPolicy Engineへ入力 ③ Allow / Approval / Denyを評価 ④ Decisionを実行フローへ反映

Next：同一Policyを異なるAgent SDKへ適用可能か確認

まず単純なファイル操作で、共通Policy適用の基本構造が実現できるかを検証する

8. 実務適用に向けた論点

基本検証を業務へ広げるには、事業側と4つの意思決定事項を具体化する必要がある

何を制御する？

File / DB / API

業務で扱う主要なリソース

Mail / Shell

送信や実行系の操作

Usage / Cost

Token, Cost, Rate, Budget
などの利用量

どの粒度で？

Tool / Action

ツール単位か、個別操作単位か

Resource / Argument

対象や引数条件まで見るか

User / Context

利用者属性や状況まで含めるか

誰が管理する？

Policy定義者

ルールを定義する担当

変更承認者

ルール変更を承認する担当

Approval担当

都度承認を行う担当

どこまで共通化？

全Agent共通

最低限の共通ルール

業務・部門別

業務特性に応じた差分

Agent / Tool固有

個別要件への対応

注：Token使用量，累積Cost，Rate，Budgetなど，Action以外の利用量もPolicy対象になり得る

「何を・どの粒度で・誰が・どこまで共通化して」Policy管理するかを具体化する